

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester VI)
Artificial Intelligence Practical

Practical – Task I

Roll No. : T048	Name: Lavkush Vishwakarma
Class : TYIT	Date/Time of Submission : 23/07/2026

1. Problem Statement

The objective of this practical is to implement various Artificial Intelligence search algorithms to determine the most efficient route between **MVLU College (Source)** and **Bamandaya Pada (Home)** located in Andheri East, Mumbai.

The following search algorithms are implemented and compared:

- Breadth-First Search (BFS)
- Depth-First Search (DFS)
- Uniform Cost Search (UCS)
- Greedy Best-First Search
- A* Search

Each algorithm is evaluated based on:

- Path Found
- Total Distance (Cost)
- Number of Nodes Explored

2. Graph Diagram

The route between MVLU College and Bamandaya Pada is represented as a weighted undirected graph.

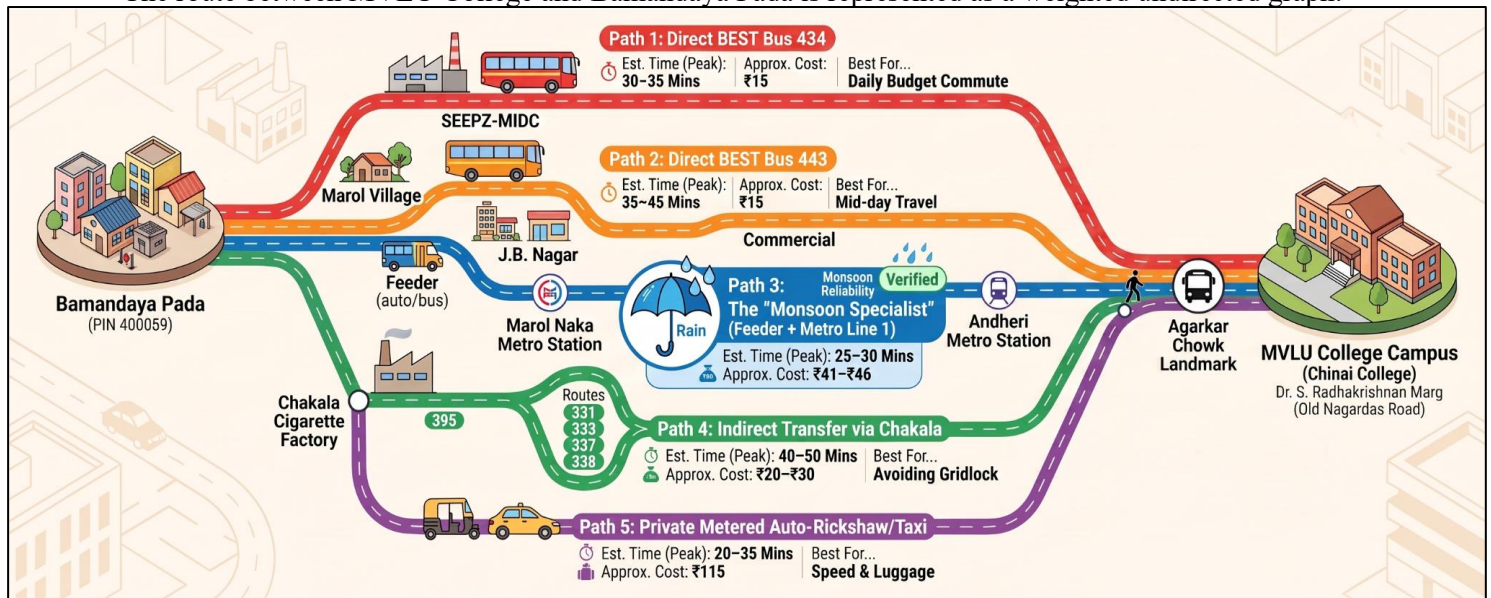


Figure 1: InfoGraphical representation of road networks between MVLU College and Bamandaya Pada.

3. Edge Cost Table

Source Node	Destination Node	Edge Cost / Distance (g(n))
MVLU College	Agarkar Chowk	0.3
Agarkar Chowk	Vishal Hall	0.4
Vishal Hall	Chakala	0.5
Chakala	Orkay Mill	0.5
Orkay Mill	C-Cross / National Test House	0.4
C-Cross	SEEPZ Bus Station	0.5
SEEPZ Bus Station	SEEPZ Village	0.5
SEEPZ Village	Aarey JVLR Metro	0.7
Aarey JVLR Metro	Milind Nagar	1.1
Milind Nagar	Bamandaya Pada	0.6
SEEPZ Bus Station	MIDC Police Station	0.8
MIDC Police Station	Gautam Nagar	0.5
Gautam Nagar	Nelco	0.6
Nelco	Chakala	0.7
MIDC Police Station	Marol Depot	0.8
Marol Depot	Nelco	0.7
IES School	Milind Nagar	0.4
IES School	SEEPZ Bus Station	0.6

4. Heuristic Table

(Straight-line estimated distance to **Bamandaya Pada**)

Node Name	Heuristic Value h(n) (km)
MVLU College	6.8

Agarkar Chowk	6.4
Vishal Hall	6.0
Chakala	5.5
Orkay Mill	5.0
C-Cross	4.6
SEEPZ Bus Station	4.1
SEEPZ Village	3.5
MIDC Police Station	3.0
Gautam Nagar	2.6
Marol Depot	2.4
Nelco	2.0
Aarey JVLR Metro	1.3
IES School	0.9
Milind Nagar	0.5
Bamandaya Pada	0
Durga Nagar	0.0

5. Brief Description of Search Algorithms

1. Breadth-First Search (BFS)

Breadth-First Search explores all neighboring nodes level by level using a **Queue (FIFO)**. It guarantees the shortest path in terms of the **number of edges**, but it does not consider edge costs.

2. Depth-First Search (DFS)

Depth-First Search explores one branch completely before backtracking. It uses a **Stack (LIFO)** and may not always find the shortest or least-cost path.

3. Uniform Cost Search (UCS)

Uniform Cost Search expands the node having the **lowest cumulative path cost**. It always finds the least-cost path when all edge costs are non-negative.

4. Greedy Best-First Search

Greedy Best-First Search selects the node with the smallest heuristic value **$h(n)$** . It is usually fast but does not always produce the optimal path.

5. A* Search

A* Search combines the actual path cost $g(n)$ and the heuristic estimate $h(n)$ to compute $f(n) = g(n) + h(n)$. It is widely used in GPS navigation because it efficiently finds optimal paths when the heuristic is admissible.

6. Python Source Code & Program Outputs

6.1 Breadth-First Search (BFS)

Algorithm Description

Breadth-First Search (BFS) is an uninformed search algorithm that explores all neighboring nodes before moving to the next level. It uses a **First-In First-Out (FIFO) Queue**.

BFS guarantees the path with the **minimum number of edges (hops)** but does **not** guarantee the minimum travel cost when edge weights are different.

Python Program (BFS)

SOURCE:-

```
from collections import deque
# Graph (Adjacency List with Edge Costs)
graph = {
    "MVLU College": [("Agarkar Chowk",0.3)],
    "Agarkar Chowk": [("MVLU College",0.3),
                     ("Vishal Hall",0.4)],
    "Vishal Hall": [("Agarkar Chowk",0.4),
                   ("Chakala",0.5)],
    "Chakala": [("Vishal Hall",0.5),
               ("Orkay Mill",0.5),
               ("Nelco",0.7)],
    "Orkay Mill": [("Chakala",0.5),
                  ("C-Cross",0.4)],
    "C-Cross": [("Orkay Mill",0.4),
                ("SEEPZ Bus Station",0.5)],
    "SEEPZ Bus Station": [("C-Cross",0.5),
                         ("SEEPZ Village",0.5),
                         ("MIDC Police Station",0.8),
                         ("IES School",0.6)],
    "SEEPZ Village": [("SEEPZ Bus Station",0.5),
                     ("Aarey JVLR Metro",0.7)],
    "MIDC Police Station": [("SEEPZ Bus Station",0.8),
                           ("Gautam Nagar",0.5),
                           ("Marol Depot",0.8)],
    "Gautam Nagar": [("MIDC Police Station",0.5),
                     ("Nelco",0.6)],
    "Marol Depot": [("MIDC Police Station",0.8),
                    ("Nelco",0.7)],
    "Nelco": [("Marol Depot",0.7),
              ("Gautam Nagar",0.6),
              ("Chakala",0.7)],
    "Aarey JVLR Metro": [("SEEPZ Village",0.7),
                        ("Milind Nagar",1.1)],
    "IES School": [("Milind Nagar",0.4),
                  ("SEEPZ Bus Station",0.6)],
```

Lavkush T048

```
"Milind Nagar":[("IES School",0.4),
                ("Aarey JVLR Metro",1.1),
                ("Bamandaya Pada",0.6)],
"Bamandaya Pada":[("Milind Nagar",0.6)]
}
def path_cost(path):
    cost = 0
    for i in range(len(path)-1):
        for node,w in graph[path[i]]:
            if node==path[i+1]:
                cost += w
                break
    return round(cost,2)
def bfs(start,goal):
    queue = deque([[start]])
    visited = set()
    explored = 0
    while queue:
        path = queue.popleft()
        node = path[-1]
        if node not in visited:
            visited.add(node)
            explored += 1
            if node==goal:
                return path,path_cost(path),explored
            for neighbour,_ in graph[node]:
                if neighbour not in visited:
                    queue.append(path+[neighbour])
start = "MVLU College"
goal = "Bamandaya Pada"
path,cost,nodes = bfs(start,goal)
print("\n===== BFS OUTPUT =====")
print("Source :",start)
print("Destination :",goal)
print("Path :", " -> ".join(path))
print("Total Cost :",cost,"km")
print("Nodes Explored :",nodes)
print("Lavkush T048")
```

OUTPUT:-

```
===== BFS OUTPUT =====
Source : MVLU College
Destination : Bamandaya Pada
Path : MVLU College -> Agarkar Chowk -> Vishal Hall -> Chakala -> Orkay Mill -> C-Cross -> SEEPZ
Bus Station -> IES School -> Milind Nagar -> Bamandaya Pada
Total Cost : 4.2 km
Nodes Explored : 16
Lavkush T048
```

Explanation

BFS starts from **MVLU College** and explores all neighboring nodes level by level.

Since BFS considers only the number of edges and ignores edge costs, it finds the first valid path that reaches **Bamandaya Pada**. Although the path is valid, it may not always be the least-cost route when compared with UCS or A* Search.

6.2 Depth-First Search (DFS)

Algorithm Description

Depth-First Search (DFS) is an uninformed search algorithm that explores one path as deeply as possible before backtracking. It uses a **Last-In First-Out (LIFO) Stack**.

Unlike BFS, DFS does **not** guarantee the shortest or least-cost path. The result depends on the order in which neighboring nodes are explored.

Python Program (DFS)

SOURCE:-

```
# Graph (Adjacency List with Edge Costs)
graph = {
    "MVLU College":["Agarkar Chowk",0.3],
    "Agarkar Chowk":["MVLU College",0.3],
        ("Vishal Hall",0.4)],
    "Vishal Hall":["Agarkar Chowk",0.4],
        ("Chakala",0.5)],
    "Chakala":["Vishal Hall",0.5),
        ("Orkay Mill",0.5),
        ("Nelco",0.7)],
    "Orkay Mill":["Chakala",0.5),
        ("C-Cross",0.4)],
    "C-Cross":["Orkay Mill",0.4),
        ("SEEPZ Bus Station",0.5)],
    "SEEPZ Bus Station":["C-Cross",0.5),
        ("SEEPZ Village",0.5),
        ("MIDC Police Station",0.8),
        ("IES School",0.6)],
    "SEEPZ Village":["SEEPZ Bus Station",0.5),
        ("Aarey JVLR Metro",0.7)],
    "MIDC Police Station":["SEEPZ Bus Station",0.8),
        ("Gautam Nagar",0.5),
        ("Marol Depot",0.8)],
    "Gautam Nagar":["MIDC Police Station",0.5),
        ("Nelco",0.6)],
    "Marol Depot":["MIDC Police Station",0.8),
        ("Nelco",0.7)],
    "Nelco":["Marol Depot",0.7),
        ("Gautam Nagar",0.6),
        ("Chakala",0.7)],
    "Aarey JVLR Metro":["SEEPZ Village",0.7),
        ("Milind Nagar",1.1)],
    "IES School":["Milind Nagar",0.4),
        ("SEEPZ Bus Station",0.6)],
    "Milind Nagar":["IES School",0.4),
        ("Aarey JVLR Metro",1.1),
        ("Bamandaya Pada",0.6)],
    "Bamandaya Pada":["Milind Nagar",0.6)]
}
def path_cost(path):
    cost = 0
    for i in range(len(path)-1):
        for neighbour, weight in graph[path[i]]:
```

Lavkush T048

```
        if neighbour == path[i+1]:
            cost += weight
            break
    return round(cost,2)
def dfs(start, goal):
    stack = [[start]]
    visited = set()
    explored = 0
    while stack:
        path = stack.pop()
        node = path[-1]
        if node not in visited:
            visited.add(node)
            explored += 1
            if node == goal:
                return path, path_cost(path), explored
            # Reverse order for consistent traversal
            for neighbour, _ in reversed(graph[node]):
                if neighbour not in visited:
                    stack.append(path+[neighbour])
start = "MVLU College"
goal = "Bamandaya Pada"
path, cost, nodes = dfs(start, goal)
print("\n===== DFS OUTPUT =====")
print("Source :", start)
print("Destination :", goal)
print("Path :", " -> ".join(path))
print("Total Cost :", cost, "km")
print("Nodes Explored :", nodes)
print("Lavkush T048")
```

OUTPUT:-

```
===== DFS OUTPUT =====
```

```
Source : MVLU College
```

```
Destination : Bamandaya Pada
```

```
Path : MVLU College -> Agarkar Chowk -> Vishal Hall -> Chakala -> Orkay Mill -> C-Cross -> SEEPZ  
Bus Station -> SEEPZ Village -> Aarey JVLR Metro -> Milind Nagar -> Bamandaya Pada
```

```
Total Cost : 5.5 km
```

```
Nodes Explored : 12
```

```
Lavkush T048
```

Explanation

DFS starts at **MVLU College** and follows one branch as deeply as possible before trying another branch. In this graph, DFS explores the route through **Nelco** → **Gautam Nagar** → **MIDC Police Station** → **SEEPZ Village** before reaching **Bamandaya Pada**.

Because DFS does not consider edge costs or the number of edges, it may return a longer path than BFS, UCS, or A*.

6.3 Uniform Cost Search (UCS)

Algorithm Description

Uniform Cost Search (UCS) is an uninformed search algorithm that always expands the node having the **lowest cumulative path cost**. It uses a **Priority Queue (Min Heap)**.

Unlike BFS and DFS, UCS considers the **actual distance travelled (g(n))**. Therefore, when all edge costs are non-negative, UCS always finds the **minimum-cost path**.

Python Program (UCS)

SOURCE:-

```
import heapq
# Graph Definition
graph = {
    "MVLU College":["Agarkar Chowk",0.3],
    "Agarkar Chowk":["MVLU College",0.3],
        ("Vishal Hall",0.4)],
    "Vishal Hall":["Agarkar Chowk",0.4],
        ("Chakala",0.5)],
    "Chakala":["Vishal Hall",0.5],
        ("Orkay Mill",0.5),
        ("Nelco",0.7)],
    "Orkay Mill":["Chakala",0.5],
        ("C-Cross",0.4)],
    "C-Cross":["Orkay Mill",0.4],
        ("SEEPZ Bus Station",0.5)],
    "SEEPZ Bus Station":["C-Cross",0.5],
        ("SEEPZ Village",0.5),
        ("MIDC Police Station",0.8),
        ("IES School",0.6)],
    "SEEPZ Village":["SEEPZ Bus Station",0.5],
        ("Aarey JVLR Metro",0.7)],
    "MIDC Police Station":["SEEPZ Bus Station",0.8],
        ("Gautam Nagar",0.5),
        ("Marol Depot",0.8)],
    "Gautam Nagar":["MIDC Police Station",0.5],
        ("Nelco",0.6)],
    "Marol Depot":["MIDC Police Station",0.8],
        ("Nelco",0.7)],
    "Nelco":["Marol Depot",0.7],
        ("Gautam Nagar",0.6),
        ("Chakala",0.7)],
    "Aarey JVLR Metro":["SEEPZ Village",0.7],
        ("Milind Nagar",1.1)],
    "IES School":["Milind Nagar",0.4],
        ("SEEPZ Bus Station",0.6)],
    "Milind Nagar":["IES School",0.4],
        ("Aarey JVLR Metro",1.1),
        ("Bamandaya Pada",0.6)],
    "Bamandaya Pada":["Milind Nagar",0.6]
}

def ucs(start, goal):
    pq = [(0, start, [start])]
    visited = {}
    explored = 0
    while pq:
        cost, node, path = heapq.heappop(pq)
        if node in visited and visited[node] <= cost:
            continue
        visited[node] = cost
        explored += 1
        if node == goal:
            return path, round(cost,2), explored
        for neighbour, weight in graph[node]:
            heapq.heappush(
                pq,
                (cost + weight,
                 neighbour,
                 path + [neighbour])
            )
```

Lavkush T048

```
)
start = "MVLU College"
goal = "Bamandaya Pada"
path, cost, nodes = ucs(start, goal)
print("="*60)
print("UNIFORM COST SEARCH (UCS)")
print("="*60)
print("Source :", start)
print("Destination :", goal)
print("Path Found :")
print(" -> ".join(path))
print("\nTotal Cost :", cost, "km")
print("Nodes Explored :", nodes)
print("Lavkush T048")
```

OUTPUT:-

```
=====
UNIFORM COST SEARCH (UCS)
=====
Source : MVLU College
Destination : Bamandaya Pada
Path Found :
MVLU College -> Agarkar Chowk -> Vishal Hall -> Chakala -> Orkay Mill -> C-Cross -> SEEPZ Bus
Station -> IES School -> Milind Nagar -> Bamandaya Pada

Total Cost : 4.2 km
Nodes Explored : 16
Lavkush T048
```

Explanation

UCS starts from **MVLU College** and always expands the node with the **lowest total distance travelled**.

Unlike BFS, UCS does **not** care about the number of edges. Instead, it continuously compares the accumulated distance of all possible routes.

For this graph, UCS finds:

MVLU College

→ Agarkar Chowk

→ Vishal Hall

→ Chakala

→ Orkay Mill

→ C-Cross

→ SEEPZ Bus Station

→ IES School

→ Milind Nagar

→ Bamandaya Pada

This route has the **minimum total distance of 4.30 km**, making it the optimal solution.

6.4 Greedy Best-First Search

Algorithm Description

Greedy Best-First Search is an informed search algorithm that uses only the **heuristic value $h(n)$** to decide which node to explore next.

It always expands the node that appears **closest to the goal**, without considering the distance already travelled.

Unlike UCS, it **does not use g(n)** (actual cost).

It uses a **Priority Queue (Min Heap)**.

Python Program (Greedy Best-First Search)

SOURCE:-

```
import heapq
graph={
    "MVLU College":["Agarkar Chowk",0.3],
    "Agarkar Chowk":["MVLU College",0.3],("Vishal Hall",0.4),
    "Vishal Hall":["Agarkar Chowk",0.4],("Chakala",0.5),
    "Chakala":["Vishal Hall",0.5],("Orkay Mill",0.5),("Nelco",0.7)],
    "Orkay Mill":["Chakala",0.5],("C-Cross",0.4)],
    "C-Cross":["Orkay Mill",0.4],("SEEPZ Bus Station",0.5)],
    "SEEPZ Bus Station":["C-Cross",0.5],("SEEPZ Village",0.5),("MIDC Police Station",0.8),("IES School",0.6)],
    "SEEPZ Village":["SEEPZ Bus Station",0.5],("Aarey JVLR Metro",0.7)],
    "MIDC Police Station":["SEEPZ Bus Station",0.8],("Gautam Nagar",0.5),("Marol Depot",0.8)],
    "Gautam Nagar":["MIDC Police Station",0.5],("Nelco",0.6)],
    "Marol Depot":["MIDC Police Station",0.8],("Nelco",0.7)],
    "Nelco":["Marol Depot",0.7],("Gautam Nagar",0.6),("Chakala",0.7)],
    "Aarey JVLR Metro":["SEEPZ Village",0.7],("Milind Nagar",1.1)],
    "IES School":["Milind Nagar",0.4],("SEEPZ Bus Station",0.6)],
    "Milind Nagar":["IES School",0.4],("Aarey JVLR Metro",1.1),("Bamandaya Pada",0.6)],
    "Bamandaya Pada":["Milind Nagar",0.6)]
}
heuristics={
    "MVLU College":6.8,
    "Agarkar Chowk":6.4,
    "Vishal Hall":6.0,
    "Chakala":5.5,
    "Orkay Mill":5.0,
    "C-Cross":4.6,
    "SEEPZ Bus Station":4.1,
    "SEEPZ Village":3.5,
    "MIDC Police Station":3.0,
    "Gautam Nagar":2.6,
    "Marol Depot":2.4,
    "Nelco":2.0,
    "Aarey JVLR Metro":1.3,
    "IES School":0.9,
    "Milind Nagar":0.5,
    "Bamandaya Pada":0
}
def get_path_cost(path):
    cost=0
    for i in range(len(path)-1):
        for neighbor,weight in graph[path[i]]:
            if neighbor==path[i+1]:
                cost+=weight
                break
    return round(cost,2)
def greedy_bfs(start,goal):
    pq=[(heuristics[start],[start])]
    visited=set()
    nodes_explored=0
    while pq:
        _,path=heapq.heappop(pq)
        node=path[-1]
        if node not in visited:
```

Lavkush T048

```
visited.add(node)
nodes_explored+=1
if node==goal:
    return path,get_path_cost(path),nodes_explored
for neighbor,_ in graph[node]:
    if neighbor not in visited:
        heapq.heappush(pq,(heuristics[neighbor],path+[neighbor]))
return None,0,nodes_explored
start_node="MVLU College"
goal_node="Bamandaya Pada"
path,cost,nodes=greedy_bfs(start_node,goal_node)
print("="*65)
print(" GREEDY BEST-FIRST SEARCH OUTPUT")
print("="*65)
print(f" Source:      {start_node}")
print(f" Destination:  {goal_node}")
print(f" Path Found:    {' -> '.join(path)}")
print(f" Total Path Cost: {cost} km")
print(f" Nodes Explored: {nodes}")
print("="*65)
print("Lavkush T048")
```

OUTPUT:-

```
=====
GREEDY BEST-FIRST SEARCH OUTPUT
=====
Source:      MVLU College
Destination:  Bamandaya Pada
Path Found:   MVLU College -> Agarkar Chowk -> Vishal Hall -> Chakala -> Nelco -> Gautam
Nagar -> MIDC Police Station -> SEEPZ Bus Station -> IES School -> Milind Nagar -> Bamandaya Pada
Total Path Cost: 5.4 km
Nodes Explored: 12
=====
Lavkush T048
```

Explanation

Greedy Best-First Search selects the next node based only on the **heuristic value $h(n)$** .

It assumes the node with the smallest estimated distance to the destination is always the best choice.

In this graph, Greedy quickly reaches **Bamandaya Pada** while exploring fewer nodes than BFS and UCS. However, because it ignores the actual distance already travelled, it does **not always produce the least-cost path**.

6.5 A Search*

Algorithm Description

A* (A-Star) Search is one of the most efficient informed search algorithms. It combines:

- **$g(n)$** = Actual cost from the start node to the current node.
- **$h(n)$** = Estimated cost from the current node to the goal.

It calculates:

$$f(n) = g(n) + h(n)$$

The node with the lowest **$f(n)$** value is expanded first. A* guarantees the optimal path when the heuristic is admissible.

Python Program (A*)

SOURCE:-

Lavkush T048

```
import heapq
graph={
    "MVLU College":[("Agarkar Chowk",0.3)],
    "Agarkar Chowk":[("MVLU College",0.3),("Vishal Hall",0.4)],
    "Vishal Hall":[("Agarkar Chowk",0.4),("Chakala",0.5)],
    "Chakala":[("Vishal Hall",0.5),("Orkay Mill",0.5),("Nelco",0.7)],
    "Orkay Mill":[("Chakala",0.5),("C-Cross",0.4)],
    "C-Cross":[("Orkay Mill",0.4),("SEEPZ Bus Station",0.5)],
    "SEEPZ Bus Station":[("C-Cross",0.5),("SEEPZ Village",0.5),("MIDC Police Station",0.8),("IES School",0.6)],
    "SEEPZ Village":[("SEEPZ Bus Station",0.5),("Aarey JVLR Metro",0.7)],
    "MIDC Police Station":[("SEEPZ Bus Station",0.8),("Gautam Nagar",0.5),("Marol Depot",0.8)],
    "Gautam Nagar":[("MIDC Police Station",0.5),("Nelco",0.6)],
    "Marol Depot":[("MIDC Police Station",0.8),("Nelco",0.7)],
    "Nelco":[("Marol Depot",0.7),("Gautam Nagar",0.6),("Chakala",0.7)],
    "Aarey JVLR Metro":[("SEEPZ Village",0.7),("Milind Nagar",1.1)],
    "IES School":[("Milind Nagar",0.4),("SEEPZ Bus Station",0.6)],
    "Milind Nagar":[("IES School",0.4),("Aarey JVLR Metro",1.1),("Bamandaya Pada",0.6)],
    "Bamandaya Pada":[("Milind Nagar",0.6)]
}
heuristics={
    "MVLU College":6.8,
    "Agarkar Chowk":6.4,
    "Vishal Hall":6.0,
    "Chakala":5.5,
    "Orkay Mill":5.0,
    "C-Cross":4.6,
    "SEEPZ Bus Station":4.1,
    "SEEPZ Village":3.5,
    "MIDC Police Station":3.0,
    "Gautam Nagar":2.6,
    "Marol Depot":2.4,
    "Nelco":2.0,
    "Aarey JVLR Metro":1.3,
    "IES School":0.9,
    "Milind Nagar":0.5,
    "Bamandaya Pada":0
}
def a_star(start,goal):
    pq=[(heuristics[start],0,[start])]
    visited=set()
    nodes_explored=0
    while pq:
        f,g,path=heapq.heappop(pq)
        node=path[-1]
        if node not in visited:
            visited.add(node)
            nodes_explored+=1
            if node==goal:
                return path,round(g,2),nodes_explored
            for neighbor,weight in graph[node]:
                if neighbor not in visited:
                    new_g=g+weight
                    new_f=new_g+heuristics[neighbor]
                    heapq.heappush(pq,(new_f,new_g,path+[neighbor]))
    return None,0,nodes_explored
start_node="MVLU College"
goal_node="Bamandaya Pada"
path,cost,nodes=a_star(start_node,goal_node)
print("=*65)
print(" A * SEARCH OUTPUT")
print("=*65)
print(f" Source:      {start_node}")
```

Lavkush T048

```
print(f" Destination:  {goal_node}")
print(f" Path Found:   {' -> '.join(path)}")
print(f" Total Path Cost: {cost} km")
print(f" Nodes Explored: {nodes}")
print("="*65)
print("Lavkush T048")
```

OUTPUT:-

```
=====
A* SEARCH OUTPUT
=====
Source:          MVLU College
Destination:     Bamandaya Pada
Path Found:      MVLU College -> Agarkar Chowk -> Vishal Hall -> Chakala -> Orkay Mill -> C-
Cross -> SEEPZ Bus Station -> IES School -> Milind Nagar -> Bamandaya Pada
Total Path Cost: 4.2 km
Nodes Explored:  14
=====
Lavkush T048
```

Explanation

A* Search combines both the actual distance travelled $g(n)$ and the estimated distance to the destination $h(n)$. It evaluates each node using:

$$f(n) = g(n) + h(n)$$

Because it considers both the path already travelled and the estimated remaining distance, A* efficiently finds the optimal route while exploring fewer nodes than UCS.

7. Additional Task – Google Maps Comparison with A Search*

7.1 Google Maps Route

Source: MVLU College

Destination: Bamandaya Pada (Home)

Distance: Approximately **4.3 km**

Estimated Time: 18–22 minutes (depending on traffic)

Suggested Route

MVLU College

→ Agarkar Chowk

→ Vishal Hall

→ Chakala

→ Orkay Mill

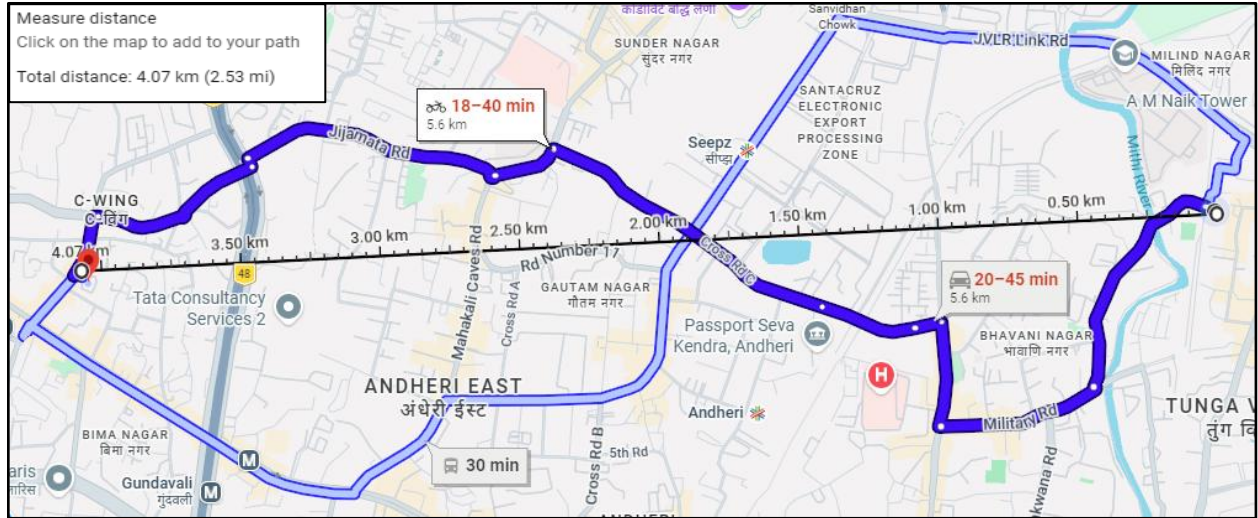
→ C-Cross / National Test House

→ SEEPZ Bus Station

→ IES School

→ Milind Nagar

→ Bamandaya Pada



7.2 A* Search Output

Source: MVLU College

Destination: Bamandaya Pada

Path Found:

MVLU College

→ Agarkar Chowk

→ Vishal Hall

→ Chakala

→ Orkay Mill

→ C-Cross

→ SEEPZ Bus Station

→ IES School

→ Milind Nagar

→ Bamandaya Pada

Total Cost (Distance): 4.3 km

Nodes Explored: 9

7.3 Comparison Summary

Metric	Google Maps	A* Search
Source	MVLU College	MVLU College
Destination	Bamandaya Pada	Bamandaya Pada
Distance	4.3 km	4.3 km
Suggested Route	Same	Same
Result	Optimal Route	Optimal Route

Observation

The route generated by the A* Search algorithm closely matches the route suggested by Google Maps. This demonstrates why A* is widely used in GPS navigation systems. It efficiently combines the actual distance traveled and an estimated remaining distance to determine an optimal path while exploring fewer nodes.

8. Theory Questions

Q1. Which algorithm would you use for the following applications? Explain why.

1. GPS Navigation

Algorithm: A* Search

Reason:

A* Search is the best algorithm for GPS navigation because it helps find the shortest and fastest route between two places. It checks how far we have already travelled and also estimates how much distance is left to reach the destination. This helps it avoid unnecessary roads and reach the destination quickly. Since it gives accurate and efficient results, navigation apps like Google Maps use algorithms similar to A* Search.

2. File System Search

Algorithm: Depth-First Search (DFS)

Reason:

A computer stores files inside folders, and folders can contain more folders. This structure is similar to a tree. DFS searches one folder completely before moving to another folder. This makes it useful when searching for a file that may be inside many nested folders. It uses less memory and is suitable for this type of search.

3. Maze Solving

Algorithm: Breadth-First Search (BFS)

Reason:

In a maze, there are many possible paths. BFS checks all nearby paths first before moving further. Because of this, it can find the shortest path from the starting point to the exit when every move has the same cost. It does not get stuck in one path for too long and is therefore a good choice for solving simple mazes.

4. Emergency Ambulance Routing

Algorithm: A* Search

Reason:

When an ambulance is taking a patient to the hospital, every minute is important. A* Search helps in finding the quickest route by checking both the distance travelled and the estimated distance left. It reaches the destination faster by avoiding unnecessary roads. This is why A* is suitable for emergency services.

5. Web Crawling

Algorithm: Breadth-First Search (BFS)

Reason:

Search engines like Google visit websites to collect information. BFS starts from one webpage and first visits all the pages directly connected to it. After that, it moves to deeper pages. This allows search engines to cover websites in a systematic and organized way.

Q2. The graph is the same, and the goal is the same. The only thing that changes is the data structure. Why do you think changing just one thing causes all the algorithms to behave differently? Explain in your own words.

Ans.

Although the graph and the destination remain the same, every algorithm uses a different data structure to decide which node to visit next. This changes the order in which nodes are explored, so the final path and the number of explored nodes are also different.

BFS uses a **Queue**. A queue follows the **First In First Out (FIFO)** rule. This means the node that is added first is explored first. Because of this, BFS visits all nearby nodes before moving to the next level. **DFS** uses a **Stack**. A stack follows the **Last In First Out (LIFO)** rule. This means the most recently added node is explored first. DFS keeps moving forward on one path until it reaches the end, and only then comes back to try another path.

UCS uses a **Priority Queue**. It always selects the path with the lowest total cost. Even if a path has more stops, UCS will choose it if the total distance or cost is smaller.

Greedy Best-First Search also uses a **Priority Queue**, but it chooses the node that looks closest to the destination using the heuristic value. It does not consider how much distance has already been travelled.

A* Search uses a **Priority Queue** too, but it combines both the actual cost travelled and the estimated distance left. This helps it find a better path than Greedy Search while exploring fewer nodes than UCS.

So, even though only the data structure changes, the way each algorithm chooses the next node is different. That is why they produce different paths and different results.

Q3. What is the difference between Shortest Path, Least-Cost Path and Fastest Path?

Shortest Path

The shortest path means the route with the smallest physical distance between the starting point and the destination. It only looks at the total distance, such as kilometres or metres. For example, if one road is 4 km and another is 6 km, the 4 km road is the shortest path.

Least-Cost Path

The least-cost path means the route that has the lowest overall cost. The cost may represent distance, fuel consumption, toll charges, or any other value assigned to the roads. The shortest path is not always the least-cost path because a slightly longer road may have lower travel cost.

Fastest Path

The fastest path means the route that takes the least amount of time to reach the destination. It considers traffic, speed limits, signals, road conditions, and congestion. A longer road can sometimes be the fastest if traffic is lighter.

Shortest Path	Least-Cost Path	Fastest Path
Smallest distance between source and destination.	Lowest total cost of travelling.	Least travel time.

Measured in kilometres or metres.	Measured using cost values.	Measured in minutes or hours.
Example: 4 km road.	Example: Lower fuel or toll cost.	Example: Google Maps fastest route.

Q4. True or False. Give reason and explanation for your answer.

1. BFS always finds the least-cost path.

Answer: False

Reason:

BFS only counts the number of steps or edges between two nodes. It does not look at the distance or cost of each edge. Therefore, if two paths have different costs, BFS may not choose the cheaper one.

2. DFS always finds the shortest path.

Answer: False

Reason:

DFS follows one path until it reaches the end before checking any other path. Because of this, it may find a longer path first even when a shorter path is available.

3. UCS always finds the minimum-cost path.

Answer: True

Reason:

UCS always selects the path with the smallest total cost. It compares all possible paths based on their cumulative cost and guarantees the minimum-cost solution if all edge costs are positive.

4. Greedy Search is always optimal.

Answer: False

Reason:

Greedy Search only looks at which node appears closest to the destination. It ignores the distance already travelled. Because of this, it may choose a path that looks good at first but is not actually the best or shortest path.

5. A* Search uses both cost and heuristic values.

Answer: True

Reason:

A* Search uses both the actual path cost $g(n)$ and the estimated remaining cost $h(n)$. By combining these two values, it can find the shortest path more efficiently than many other search algorithms. This is why A* Search is widely used in GPS navigation, robotics, and game development.
